Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №5

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

«Розробка багатовіконного інтерфейсу користувача

для графічного редактора об’єктів»

Виконав:

студент групи ІМ-52
Пращук Данило Павлович
Номер в списку - 16

Перевірив:

Рекечинський Д. О.

Київ 2026

Завдання

1. Створити у середовищі розробки проект з ім’ям Lab5.

2. Написати вихідний текст програми згідно варіанту завдання.

3. Скомпілювати вихідний текст і отримати виконуваний файл програми.

4. Перевірити роботу програми. За потреби налагодити програму.

5. Проаналізувати та прокоментувати результати та вихідний текст програми.

6. Оформити звіт.

Варіанти завдань та основні вимоги

1. Номер варіанту завдання дорівнює номеру зі списку студентів у журналі.

Студенти з непарним номером (1, 3, 5, . . .) програмують глобальний статичний об'єкт класу MyEditor у вигляді Singleton Меєрса.

Студенти з парним номером (2, 4, 6, . . .) програмують об’єкт класу MyEditor на основі класичної реалізації Singleton.

2. Усі кольори та стилі геометричних форм – як у попередній лабораторній роботі №4.

3. Запрограмувати вікно таблиці. Для його відкриття та закриття передбачити окремий пункт меню. Вікно таблиці повинно автоматично закриватися при виході з програми.

4. Вікно таблиці – немодальне вікно діалогу. Таблиця повинна бути запрограмована як клас у окремому модулі. Інтерфейс модуля у вигляді оголошення класу таблиці

5. Запрограмувати запис файлу множини об’єктів, що вводяться

6. Ієрархія класів та побудова модулів повинні бути зручними для

можливостей додавання нових типів об'єктів без переписування коду вже існуючих модулів.

7. У звіті повинна бути схема успадкування класів – діаграма класів. Для побудови діаграми класів дозволяється обрати інструменти на власний розсуд.

8. Бонуси-заохочення, які можуть суттєво підвищити оцінку лабораторної роботи. Оцінка підвищується за виконання кожного пункту, з наведених нижче:

1) Якщо у вікні таблиці буде передбачено, щоб користувач міг виділити курсором рядок таблиці і відповідний об’єкт буде якось виділятися на зображенні у головному вікні.

2) Якщо у вікні таблиці користувач може виділити курсором рядок таблиці і відповідний об’єкт буде вилучено з масиву об’єктів.

3) Якщо програма не тільки записує у файл опис множини об’єктів, а ще й здатна завантажити такий файл і відобразити відповідні об’єкти у головному вікні та вікні таблиці.

При виконанні бонусів 1 та 2 забороняється робити для цього нові залежності модуля my_table від інших .xxx файлів. Тоді як надіслати повідомлення (наприклад, про виділення користувачем якогось рядка таблиці) від вікна таблиці клієнту цього вікна (наприклад, коду головного файлу)?

Підказки можна знайти у матеріалі лекції стосовно технології Callback, а також патернів Observer, Listener.

main.cpp

```
#include <QApplication>
#include <QMainWindow>
#include "mainWindow.h"

int main(int argc, char *argv[])
{
   QApplication app(argc, argv);
```

```
    MainWindow window;


    window.show();
    return app.exec();
}
```

mainWindow.cpp

```
#include "mainWindow.h"
#include <QMenu>
#include <QMenuBar>
#include <QToolBar>
#include <QIcon>
#include <QActionGroup>
#include <QMessageBox>
#include <QFileDialog>

MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent)
{
    setWindowTitle("Lab 5");
    resize(640, 480);

    editor = MyEditor::getInstance(this);
    setCentralWidget(editor);

    QMenuBar *menu = menuBar();
    QMenu *menuFile = menu->addMenu("&Файл");

    QAction *fileAction = menuFile->addAction("Очистити");
    QAction *tableAction = menuFile->addAction("Таблиця фігур");
    QAction *saveAction = menuFile->addAction("Зберегти у файл");
```

```
QAction *loadAction = menuFile->addAction("Завантажити з файлу");

toolBar = addToolBar("Панель фігур");
toolBar->setMovable(false);

QMenu *menuObjects = menu->addMenu("&Об’єкти");
QActionGroup *shapeGroup = new QActionGroup(this);
shapeGroup->setExclusive(true);

struct ToolInfo
{
   QString iconPath;
   QString name;
   QString toolTip;
   int shapeType;
};

const QList<ToolInfo> tools = {
   {":icons/point.png", "Крапка", "Інструмент: Крапка", ShapePoint},
   {":icons/line.png", "Лінія", "Інструмент: Лінія", ShapeLine},
   {":icons/rectangle.png", "Прямокутник", "Інструмент: Прямокутник (від центру)", ShapeRect},
   {":icons/ellipse.png", "Еліпс", "Інструмент: Еліпс", ShapeEllipse},
   {":icons/linedots.png", "Лінія з крапками", "Інструмент: Лінія з крапками", ShapeLineDots},
   {":icons/cube.png", "Каркас куба", "Інструмент: Каркас куба", ShapeCube}};

QAction *defaultAction = nullptr;

for (const auto &info : tools)
{
```

```
    QAction *action = new QAction(QIcon(info.iconPath), info.name, this);
    action->setCheckable(true);
    action->setToolTip(info.toolTip);
    action->setData(info.shapeType);

    shapeGroup->addAction(action);
    toolBar->addAction(action);
    menuObjects->addAction(action);

    if (info.shapeType == ShapePoint)
    {
       defaultAction = action;
    }
  }

  QMenu *menuInfo = menu->addMenu("&Довідка");
  QAction *infoAction = menuInfo->addAction("Довідкова інформація");

  connect(fileAction, &QAction::triggered, editor, &MyEditor::clearCanvas);

  connect(saveAction, &QAction::triggered, this, [this]()
      {
    QString fileName = QFileDialog::getSaveFileName(this, "Зберегти файл", "", "Text Files (*.txt);;All
Files (*)");
    if (!fileName.isEmpty()) {
       editor->saveToFile(fileName);
    } });

  connect(loadAction, &QAction::triggered, this, [this]()
```

```
    {
    QString fileName = QFileDialog::getOpenFileName(this, "Відкрити файл", "", "Text Files (*.txt);;All Files (*)");
    if (!fileName.isEmpty()) {
        editor->loadFromFile(fileName);
    } });

  connect(tableAction, &QAction::triggered, this, [this]()
    {
    if (!tableWindow) {
        tableWindow = new MyTableWindow(this);
    }

    tableWindow->setOnRowSelectedCallback([this](int index) {
        editor->selectShapeByIndex(index);
    });
    tableWindow->setGetCountCallback([this]() {
        return editor->getShapeCount();
    });
    tableWindow->setDataCallback([this](int index, QString &name, QString &coords) {
        Shape *shape = editor->getShape(index);
        if (shape) {
            name = shape->getShapeName();
            coords = shape->getCoordinates();
        }
    });
    tableWindow->setOnHiddenCallback([this]() {
        editor->selectShapeByIndex(-1);
    });
```

```
    editor->setOnShapesChangedCallback([this]() {
        if (tableWindow) {
            tableWindow->updateTableData();
        }
    });
    tableWindow->setOnDeleteRowCallback([this](int index) {
            editor->deleteShapeByIndex(index);
            tableWindow->updateTableData();
        });

    tableWindow->show();
    tableWindow->raise();
    tableWindow->activateWindow(); });

  connect(infoAction, &QAction::triggered, this, [this]()
      { QMessageBox::information(this, "Довідка",
                  "Лабораторна робота №5 «Розробка багатовіконного інтерфейсу користувача для графічного редактора об’єктів».<br>"
                  "Варіант: 16<br>"
                  "Автор: Пращук Данило Павлович ІМ-52"); });

  connect(shapeGroup, &QActionGroup::triggered, this, &MainWindow::onNotify);

  if (defaultAction)
  {
    defaultAction->setChecked(true);
    onNotify(defaultAction);
  }
```

```
}

MainWindow::~MainWindow()
{
    MyEditor::destroyInstance();
}

void MainWindow::onNotify(QAction *action)
{
    int shapeType = action->data().toInt();
    QString shapeName;
    Shape *newShape = nullptr;

    switch (shapeType)
    {
    case ShapePoint:
        newShape = new PointShape();
        shapeName = "Крапка";
        break;
    case ShapeLine:
        newShape = new LineShape();
        shapeName = "Лінія";
        break;
    case ShapeRect:
        newShape = new RectShape();
        shapeName = "Прямокутник";
        break;
    case ShapeEllipse:
        newShape = new EllipseShape();
```

```
        shapeName = "Еліпс";
        break;
    case ShapeLineDots:
        newShape = new LineDotsShape();
        shapeName = "Лінія з крапками";
        break;
    case ShapeCube:
        newShape = new CubeShape();
        shapeName = "Каркас куба";
        break;
    default:
        break;
    }

    if (newShape)
    {
        editor->setCurrentShape(newShape);
        setWindowTitle("Lab 5 - " + shapeName);
    }
}
```

mainWindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include "myEditor.h"
```

```
#include "myTable.h"

class QToolBar;

class MainWindow : public QMainWindow
{
   Q_OBJECT

public:
   MainWindow(QWidget *parent = nullptr);
   ~MainWindow();



private slots:
   void onNotify(QAction *action);
private:
MyEditor *editor;
QToolBar *toolBar;
MyTableWindow* tableWindow = nullptr;

};

#endif
```

MyEditor.cpp

```
#include "myEditor.h"
#include <QMouseEvent>
#include <QFile>
#include <QTextStream>
```

```
#include <QMessageBox>

MyEditor* MyEditor::instance = nullptr;

MyEditor::MyEditor(QWidget *parent)
    : QWidget(parent), shapeCount(0), currentShape(nullptr), isDragging(false)
{
}

MyEditor::~MyEditor() {
    delete currentShape;
    clearCanvas();
}

void MyEditor::setCurrentShape(Shape *shape) {
    delete currentShape;
    currentShape = shape;
}

Shape* MyEditor::createShapeByType(const QString &type) {
    if (type == "Rect")      return new RectShape();
    if (type == "Line")      return new LineShape();
    if (type == "Ellipse")   return new EllipseShape();
    if (type == "Point")     return new PointShape();
    if (type == "LineDots")  return new LineDotsShape();
    if (type == "Cube")      return new CubeShape();
    return nullptr;
}
```

```
void MyEditor::saveToFile(const QString& filename) {
    QFile file(filename);
    if (!file.open(QIODevice::WriteOnly | QIODevice::Text))
        return;

    QTextStream out(&file);
    for (int i = 0; i < shapeCount; i++) {
        if (pcshape[i]) {
            out << pcshape[i]->getInfo() << '\n';
        }
    }
}

void MyEditor::loadFromFile(const QString& filename) {
    QFile file(filename);
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text))
        return;

    clearCanvas();
    QTextStream in(&file);

    while (!in.atEnd() && shapeCount < N) {
        QString line = in.readLine().trimmed();
        if (line.isEmpty()) continue;

        QStringList parts = line.split(' ');
        Shape* newShape = createShapeByType(parts[0]);

        if (newShape) {
```

```
            newShape->createFromInfo(line);
            pcshape[shapeCount++] = newShape;
        }
    }
    redrawCanvas();
}

void MyEditor::selectShapeByIndex(int index) {
    for (int i = 0; i < shapeCount; i++) {
        if (pcshape[i]) {
            pcshape[i]->setSelected(i == index);
        }
    }
    redrawCanvas();
}

void MyEditor::deleteShapeByIndex(int index) {
    if (index < 0 || index >= shapeCount) {
        return;
    }
    delete pcshape[index];

    for (int i = index; i < shapeCount - 1; ++i) {
        pcshape[i] = pcshape[i + 1];
    }

    pcshape[shapeCount - 1] = nullptr;
    shapeCount--;
```

```
    selectShapeByIndex(-1);
    redrawCanvas();

    if (onShapesChangedCallback) {
        onShapesChangedCallback();
    }
}

void MyEditor::clearCanvas() {
    for (int i = 0; i < shapeCount; i++) {
        delete pcshape[i];
        pcshape[i] = nullptr;
    }
    shapeCount = 0;
    redrawCanvas();
}

void MyEditor::redrawCanvas() {
    pixmap.fill(Qt::white);

    {
        QPainter painter(&pixmap);
        for (int i = 0; i < shapeCount; i++) {
            if (pcshape[i]) {
                pcshape[i]->show(painter);
            }
        }
    }
```

```
    if (onShapesChangedCallback) {
        onShapesChangedCallback();
    }

    update();
}

void MyEditor::mousePressEvent(QMouseEvent *event) {
    if (event->button() == Qt::LeftButton && currentShape && shapeCount < N) {
        startPt = endPt = event->pos();
        isDragging = true;

        if (typeid(*currentShape) == typeid(PointShape)) {
            pcshape[shapeCount] = currentShape->clone();
            pcshape[shapeCount]->set(startPt.x(), startPt.y(), startPt.x(), startPt.y());
            shapeCount++;
            isDragging = false;
            redrawCanvas();
        }
    }
}

void MyEditor::mouseMoveEvent(QMouseEvent *event) {
    if (isDragging) {
        endPt = event->pos();
        update();
    }
}
```

```
void MyEditor::mouseReleaseEvent(QMouseEvent *event) {
    if (event->button() == Qt::LeftButton && isDragging) {
        isDragging = false;
        endPt = event->pos();

        if (currentShape && shapeCount < N) {
            pcshape[shapeCount] = currentShape->clone();
            pcshape[shapeCount]->set(startPt.x(), startPt.y(), endPt.x(), endPt.y());
            shapeCount++;
            redrawCanvas();
        }
    }
}

void MyEditor::resizeEvent(QResizeEvent *event) {
    QWidget::resizeEvent(event);
    pixmap = QPixmap(size());
    redrawCanvas();
}

void MyEditor::paintEvent(QPaintEvent *event) {
    Q_UNUSED(event);
    QPainter painter(this);

    painter.drawPixmap(0, 0, pixmap);

    if (isDragging && currentShape) {
        currentShape->set(startPt.x(), startPt.y(), endPt.x(), endPt.y());
        currentShape->rubber(painter);
```

```
    }
}
```

myEditor.h

```
#ifndef MYEDITOR_H
#define MYEDITOR_H

#include <QWidget>
#include <QPixmap>
#include <QPoint>
#include <functional>
#include "shape.h"

class MyEditor : public QWidget {
    Q_OBJECT

private:
    static MyEditor* instance;
    MyEditor(QWidget *parent = nullptr);
    MyEditor(const MyEditor&) = delete;
    MyEditor& operator=(const MyEditor&) = delete;
    ~MyEditor();

    static const int N = 117;
    Shape *pcshape[N] = {nullptr};
    int shapeCount;
    Shape *currentShape;

    QPixmap pixmap;
    bool isDragging;
```

```
    QPoint startPt;
    QPoint endPt;

    std::function<void()> onShapesChangedCallback;

    Shape* createShapeByType(const QString &type);

public:
    static MyEditor* getInstance(QWidget *parent = nullptr) {
        if (!instance) {
            instance = new MyEditor(parent);
        }
        return instance;
    }

    static void destroyInstance() {
        delete instance;
        instance = nullptr;
    }

    void setCurrentShape(Shape *shape);
    void clearCanvas();

    int getShapeCount() const { return shapeCount; }

    Shape* getShape(int index) const {
        if (index >= 0 && index < shapeCount) {
            return pcshape[index];
        }
```

```
        return nullptr;
    }

    void saveToFile(const QString& filename);
    void loadFromFile(const QString& filename);
    void selectShapeByIndex(int index);
    void setOnShapesChangedCallback(std::function<void()> callback) {
        onShapesChangedCallback = callback;
    }
    void deleteShapeByIndex(int index);

protected:
    void paintEvent(QPaintEvent *event) override;
    void mousePressEvent(QMouseEvent *event) override;
    void mouseMoveEvent(QMouseEvent *event) override;
    void mouseReleaseEvent(QMouseEvent *event) override;
    void resizeEvent(QResizeEvent *event) override;
    void redrawCanvas();
};

#endif
```

myTable.cpp

```
#include "myTable.h"
#include <QHeaderView>
#include <QTableWidget>
#include <QVBoxLayout>
#include <QPushButton>

MyTableWindow::MyTableWindow(QWidget *parent) : QDialog(parent) {
```

```
setWindowTitle("Таблиця об'єктів");
resize(450, 300);

QVBoxLayout *layout = new QVBoxLayout(this);
tableWidget = new QTableWidget(this);

tableWidget->setColumnCount(3);
tableWidget->setHorizontalHeaderLabels({"№", "Тип фігури", "Координати"});
tableWidget->horizontalHeader()->setStretchLastSection(true);

layout->addWidget(tableWidget);

QVBoxLayout *buttonLayout = new QVBoxLayout();
deleteButton = new QPushButton("Видалити", this);
buttonLayout->addWidget(deleteButton);

layout->addLayout(buttonLayout);
connect(deleteButton, &QPushButton::clicked, this, [this](){
   if (!onDeleteRowCallback) return;
   int row = tableWidget->currentRow();
   if (row >= 0) {
      onDeleteRowCallback(row);
   }

});

connect(tableWidget, &QTableWidget::itemSelectionChanged, this, [this]() {
   if (!onRowSelectedCallback) return;
```

```
        QList<QTableWidgetItem*> selectedItems = tableWidget->selectedItems();
        int row = selectedItems.isEmpty() ? -1 : tableWidget->row(selectedItems.first());
        onRowSelectedCallback(row);
    });
}

void MyTableWindow::updateTableData() {
    if (!getCountCallback || !getDataCallback) {
        tableWidget->setRowCount(0);
        return;
    }

    tableWidget->blockSignals(true);
    int count = getCountCallback();

    tableWidget->clearSelection();
    tableWidget->setRowCount(count);

    for (int i = 0; i < count; ++i) {
        QString name, coords;
        getDataCallback(i, name, coords);

        tableWidget->setItem(i, 0, new QTableWidgetItem(QString::number(i + 1)));

            tableWidget->setItem(i, 1, new QTableWidgetItem(name));
            tableWidget->setItem(i, 2, new QTableWidgetItem(coords));
    }
    tableWidget->blockSignals(false);
}
```

```
void MyTableWindow::showEvent(QShowEvent *event) {
    QDialog::showEvent(event);
    updateTableData();

    if (onRowSelectedCallback) {
        QList<QTableWidgetItem*> selectedItems = tableWidget->selectedItems();
        int row = selectedItems.isEmpty() ? -1 : tableWidget->row(selectedItems.first());
        onRowSelectedCallback(row);
    }
}

void MyTableWindow::hideEvent(QHideEvent *event) {
    QDialog::hideEvent(event);
    if (onHiddenCallback) {
        onHiddenCallback();
    }
}
```

myTable.h

```
#ifndef MYTABLE_H
#define MYTABLE_H

#include <QDialog>
#include <QTableWidget>
#include <functional>

class QPushButton;
class MyTableWindow : public QDialog {
    Q_OBJECT
```

```
private:
    QTableWidget* tableWidget;
    std::function<void(int)> onRowSelectedCallback;
    std::function<void(int, QString&, QString&)> getDataCallback;
    std::function<int()> getCountCallback;
    std::function<void()> onHiddenCallback;
    std::function<void(int)> onDeleteRowCallback;
    QPushButton *deleteButton;

public:
    explicit MyTableWindow(QWidget* parent = nullptr);

    void updateTableData();

    void setOnRowSelectedCallback(std::function<void(int)> callback) {
        onRowSelectedCallback = callback;
    }
    void setDataCallback(std::function<void(int index, QString &name, QString &coords)> cb) {
getDataCallback = cb; }

    void setGetCountCallback(std::function<int()> cb) { getCountCallback = cb; }
    void setOnHiddenCallback(std::function<void()> cb) { onHiddenCallback = cb; }
    void setOnDeleteRowCallback(std::function<void(int)> cb) { onDeleteRowCallback = cb; }

protected:
    void showEvent(QShowEvent* event) override;
    void hideEvent(QHideEvent* event) override;
};
```

```
#endif
```

shape.cpp

```
#include "shape.h"
#include <cmath>
#include <QPen>
#include <QColor>

Shape::Shape() : xs1(0), ys1(0), xs2(0), ys2(0) {}

void Shape::set(long x1, long y1, long x2, long y2)
{
   xs1 = x1;
   ys1 = y1;
   xs2 = x2;
   ys2 = y2;
}

void Shape::rubber(QPainter &painter)
{
   painter.setPen(QPen(Qt::red, 1, Qt::DashLine));
   painter.setBrush(Qt::NoBrush);
}



void PointShape::show(QPainter &painter)
{
   QPen pen(isSelected ? Qt::blue : Qt::black, 6, Qt::SolidLine, Qt::RoundCap);
   painter.setPen(pen);
```

```
    painter.drawPoint(xs1, ys1);
}

Shape *PointShape::clone() const { return new PointShape(*this); }

QString PointShape::getCoordinates() const {
    return QString("x: %1, y: %2").arg(xs1).arg(ys1);
}

QString PointShape::getInfo() const {
    return QString("Point %1 %2").arg(xs1).arg(ys1);
}

void PointShape::createFromInfo(const QString& info) {
    QStringList parts = info.split(" ");
    if (parts.size() == 3 && parts[0] == "Point") {
        xs1 = parts[1].toLong();
        ys1 = parts[2].toLong();
    }
}

void LineShape::show(QPainter &painter)
{
    QPen pen(isSelected ? Qt::blue : Qt::black, 2, Qt::SolidLine);
    painter.setPen(pen);
    painter.drawLine(xs1, ys1, xs2, ys2);
}

void LineShape::rubber(QPainter &painter)
```

```
{
   Shape::rubber(painter);
   painter.drawLine(xs1, ys1, xs2, ys2);
}

Shape *LineShape::clone() const { return new LineShape(*this); }

QString LineShape::getCoordinates() const {
   return QString("(%1, %2) -> (%3, %4)").arg(xs1).arg(ys1).arg(xs2).arg(ys2);
}

QString LineShape::getInfo() const {
   return QString("Line %1 %2 %3 %4").arg(xs1).arg(ys1).arg(xs2).arg(ys2);
}

void LineShape::createFromInfo(const QString& info) {
   QStringList parts = info.split(" ");
   if (parts.size() == 5 && parts[0] == "Line") {
      xs1 = parts[1].toLong();
      ys1 = parts[2].toLong();
      xs2 = parts[3].toLong();
      ys2 = parts[4].toLong();
   }
}



void RectShape::show(QPainter &painter)
{
   QPen pen(isSelected ? Qt::blue : Qt::black, 2, Qt::SolidLine);
```

```
    painter.setPen(pen);
    painter.setBrush(getBrush());

    int dx = std::abs(xs2 - xs1);
    int dy = std::abs(ys2 - ys1);
    QRect previewRect(xs1 - dx, ys1 - dy, 2 * dx, 2 * dy);
    painter.drawRect(previewRect.normalized());
}

void RectShape::rubber(QPainter &painter)
{
    Shape::rubber(painter);

    int dx = std::abs(xs2 - xs1);
    int dy = std::abs(ys2 - ys1);
    QRect previewRect(xs1 - dx, ys1 - dy, 2 * dx, 2 * dy);

    painter.drawRect(previewRect.normalized());
}

Shape *RectShape::clone() const { return new RectShape(*this); }

QString RectShape::getCoordinates() const {
    return QString("(%1, %2) -> (%3, %4)").arg(xs1).arg(ys1).arg(xs2).arg(ys2);
}

QString RectShape::getInfo() const {
    return QString("Rect %1 %2 %3 %4").arg(xs1).arg(ys1).arg(xs2).arg(ys2);
}
```

```
void RectShape::createFromInfo(const QString& info) {
    QStringList parts = info.split(" ");
    if (parts.size() == 5 && parts[0] == "Rect") {
        xs1 = parts[1].toLong();
        ys1 = parts[2].toLong();
        xs2 = parts[3].toLong();
        ys2 = parts[4].toLong();
    }
}

void EllipseShape::show(QPainter &painter)
{
    QPen pen(isSelected ? Qt::blue : Qt::black, 2, Qt::SolidLine);
    painter.setPen(pen);
    painter.setBrush(Qt::NoBrush);

    QRect previewRect(QPoint(xs1, ys1), QPoint(xs2, ys2));
    painter.drawEllipse(previewRect.normalized());
}

void EllipseShape::rubber(QPainter &painter)
{
    Shape::rubber(painter);
    QRect previewRect(QPoint(xs1, ys1), QPoint(xs2, ys2));
    painter.drawEllipse(previewRect.normalized());
}

Shape *EllipseShape::clone() const { return new EllipseShape(*this); }
```

```
QString EllipseShape::getCoordinates() const {
    return QString("(%1, %2) -> (%3, %4)").arg(xs1).arg(ys1).arg(xs2).arg(ys2);
}

QString EllipseShape::getInfo() const {
    return QString("Ellipse %1 %2 %3 %4").arg(xs1).arg(ys1).arg(xs2).arg(ys2);
}

void EllipseShape::createFromInfo(const QString& info) {
    QStringList parts = info.split(" ");
    if (parts.size() == 5 && parts[0] == "Ellipse") {
        xs1 = parts[1].toLong();
        ys1 = parts[2].toLong();
        xs2 = parts[3].toLong();
        ys2 = parts[4].toLong();
    }
}



void LineDotsShape::show(QPainter &painter) {
    LineShape::show(painter);

    int rx = 6, ry = 6;
    EllipseShape dot1;
    dot1.set(xs1 - rx, ys1 - ry, xs1 + rx, ys1 + ry);
    dot1.show(painter);

    EllipseShape dot2;
```

```
    dot2.set(xs2 - rx, ys2 - ry, xs2 + rx, ys2 + ry);
    dot2.show(painter);
}

void LineDotsShape::rubber(QPainter &painter) {
    LineShape::rubber(painter);

    int rx = 6, ry = 6;
    EllipseShape dot1;
    dot1.set(xs1 - rx, ys1 - ry, xs1 + rx, ys1 + ry);
    dot1.rubber(painter);

    EllipseShape dot2;
    dot2.set(xs2 - rx, ys2 - ry, xs2 + rx, ys2 + ry);
    dot2.rubber(painter);
}

Shape *LineDotsShape::clone() const { return new LineDotsShape(*this); }

QString LineDotsShape::getCoordinates() const {
    return QString("(%1, %2) -> (%3, %4)").arg(xs1).arg(ys1).arg(xs2).arg(ys2);
}

QString LineDotsShape::getInfo() const {
    return QString("LineDots %1 %2 %3 %4").arg(xs1).arg(ys1).arg(xs2).arg(ys2);
}

void LineDotsShape::createFromInfo(const QString& info) {
    QStringList parts = info.split(" ");
```

```
    if (parts.size() == 5 && parts[0] == "LineDots") {
        xs1 = parts[1].toLong();
        ys1 = parts[2].toLong();
        xs2 = parts[3].toLong();
        ys2 = parts[4].toLong();
    }
}


void CubeShape::show(QPainter &painter) {
    long origX1 = xs1, origY1 = ys1, origX2 = xs2, origY2 = ys2;

    int dx = std::abs(origX2 - origX1);
    int dy = std::abs(origY2 - origY1);

    long fx1 = origX1 - dx;
    long fy1 = origY1 - dy;
    long fx2 = origX1 + dx;
    long fy2 = origY1 + dy;

    int shiftX = dx / 2;
    int shiftY = dy / 2;

    long bx1 = fx1 + shiftX;
    long by1 = fy1 + shiftY;
    long bx2 = fx2 + shiftX;
    long by2 = fy2 + shiftY;

    RectShape::set(origX1, origY1, origX2, origY2);
```

```
    RectShape::show(painter);

    long bCenterX = (bx1 + bx2) / 2;
    long bCenterY = (by1 + by2) / 2;
    long bRadiusX = std::abs(bx2 - bx1) / 2;
    long bRadiusY = std::abs(by2 - by1) / 2;
    RectShape::set(bCenterX, bCenterY, bCenterX + bRadiusX, bCenterY + bRadiusY);
    RectShape::show(painter);

    LineShape::set(fx1, fy1, bx1, by1);
    LineShape::show(painter);

    LineShape::set(fx2, fy1, bx2, by1);
    LineShape::show(painter);

    LineShape::set(fx1, fy2, bx1, by2);
    LineShape::show(painter);

    LineShape::set(fx2, fy2, bx2, by2);
    LineShape::show(painter);

    RectShape::set(origX1, origY1, origX2, origY2);
    LineShape::set(origX1, origY1, origX2, origY2);
}

void CubeShape::rubber(QPainter &painter) {
    long origX1 = xs1, origY1 = ys1, origX2 = xs2, origY2 = ys2;

    int dx = std::abs(origX2 - origX1);
```

```
int dy = std::abs(origY2 - origY1);

long fx1 = origX1 - dx;
long fy1 = origY1 - dy;
long fx2 = origX1 + dx;
long fy2 = origY1 + dy;

int shiftX = dx / 2;
int shiftY = dy / 2;

long bx1 = fx1 + shiftX;
long by1 = fy1 + shiftY;
long bx2 = fx2 + shiftX;
long by2 = fy2 + shiftY;

RectShape::set(origX1, origY1, origX2, origY2);
RectShape::rubber(painter);

long bCenterX = (bx1 + bx2) / 2;
long bCenterY = (by1 + by2) / 2;
long bRadiusX = std::abs(bx2 - bx1) / 2;
long bRadiusY = std::abs(by2 - by1) / 2;
RectShape::set(bCenterX, bCenterY, bCenterX + bRadiusX, bCenterY + bRadiusY);
RectShape::rubber(painter);

LineShape::set(fx1, fy1, bx1, by1);
LineShape::rubber(painter);

LineShape::set(fx2, fy1, bx2, by1);
```

```
    LineShape::rubber(painter);

    LineShape::set(fx1, fy2, bx1, by2);
    LineShape::rubber(painter);

    LineShape::set(fx2, fy2, bx2, by2);
    LineShape::rubber(painter);

    RectShape::set(origX1, origY1, origX2, origY2);
    LineShape::set(origX1, origY1, origX2, origY2);
}

Shape *CubeShape::clone() const { return new CubeShape(*this); }

QString CubeShape::getCoordinates() const {
    return QString("(%1, %2) -> (%3, %4)").arg(xs1).arg(ys1).arg(xs2).arg(ys2);
}

QString CubeShape::getInfo() const {
    return QString("Cube %1 %2 %3 %4").arg(xs1).arg(ys1).arg(xs2).arg(ys2);
}

void CubeShape::createFromInfo(const QString& info) {
    QStringList parts = info.split(" ");
    if (parts.size() == 5 && parts[0] == "Cube") {
        xs1 = parts[1].toLong();
        ys1 = parts[2].toLong();
        xs2 = parts[3].toLong();
        ys2 = parts[4].toLong();
```

```
    }
}
```

shape.h

```
#ifndef SHAPE_H
#define SHAPE_H

#include <QPainter>
#include <QRect>
#include <QPoint>
#include <QBrush>
#include <QColor>

enum ShapeType {
    ShapePoint,
    ShapeLine,
    ShapeRect,
    ShapeEllipse,
    ShapeLineDots,
    ShapeCube
};

class Shape {
protected:
    long xs1, ys1, xs2, ys2;
    bool isSelected = false;

public:
    Shape();
```

```
    virtual ~Shape() = default;

    virtual void setSelected(bool selected) { isSelected = selected; }
    virtual void set(long x1, long y1, long x2, long y2);

    virtual void show(QPainter &painter) = 0;
    virtual void rubber(QPainter &painter);
    virtual Shape* clone() const = 0;

    virtual QString getInfo() const = 0;
    virtual void createFromInfo(const QString& info) = 0;
    virtual QString getCoordinates() const = 0;
    virtual QString getShapeName() const = 0;
};

class PointShape : public Shape {
public:
    void show(QPainter &painter) override;
    Shape* clone() const override;
    QString getInfo() const override;
    void createFromInfo(const QString& info) override;
    QString getCoordinates() const override;
    QString getShapeName() const override { return "Крапка"; }
};

class LineShape : virtual public Shape {
public:
    void show(QPainter &painter) override;
    void rubber(QPainter &painter) override;
```

```
    Shape* clone() const override;
    QString getInfo() const override;
    void createFromInfo(const QString& info) override;
    QString getCoordinates() const override;
    QString getShapeName() const override { return "Лінія"; }
};

class RectShape : virtual public Shape {
public:
    void show(QPainter &painter) override;
    void rubber(QPainter &painter) override;
    Shape* clone() const override;

    virtual QBrush getBrush() {
        return QColor(128, 128, 128);
    }

    QString getInfo() const override;
    void createFromInfo(const QString& info) override;
    QString getCoordinates() const override;
    QString getShapeName() const override { return "Прямокутник"; }
};

class EllipseShape : virtual public Shape {
public:
    void show(QPainter &painter) override;
    void rubber(QPainter &painter) override;
    Shape* clone() const override;
    QString getInfo() const override;
```

```
    void createFromInfo(const QString& info) override;
    QString getCoordinates() const override;
    QString getShapeName() const override { return "Еліпс"; }
};

class LineDotsShape : public LineShape, public EllipseShape {
public:
    void show(QPainter &painter) override;
    void rubber(QPainter &painter) override;
    Shape* clone() const override;
    QString getInfo() const override;
    void createFromInfo(const QString& info) override;
    QString getCoordinates() const override;
    QString getShapeName() const override { return "Лінія з кругами"; }
};

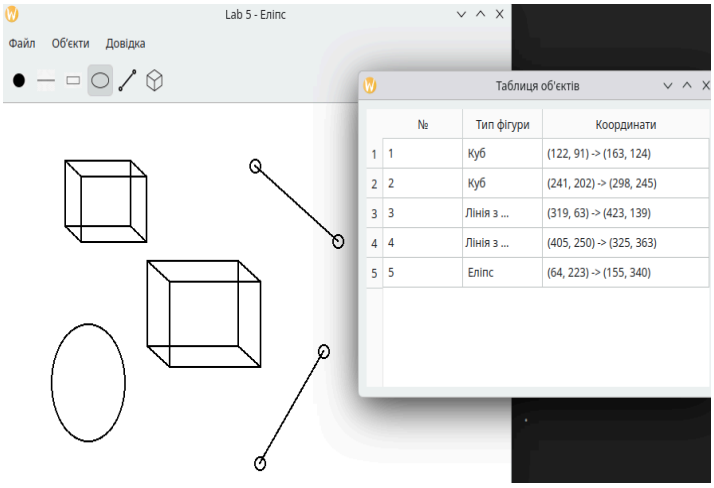
class CubeShape : public LineShape, public RectShape {
public:
    void show(QPainter &painter) override;
    void rubber(QPainter &painter) override;
    Shape* clone() const override;

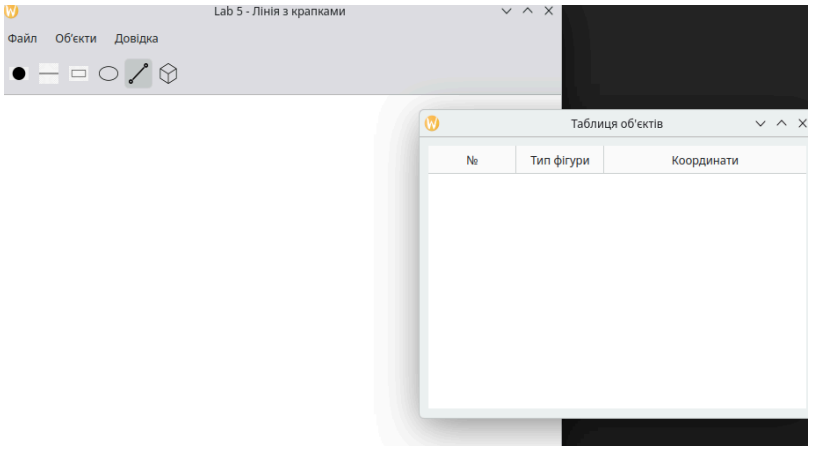
    QBrush getBrush() override { return Qt::NoBrush; }

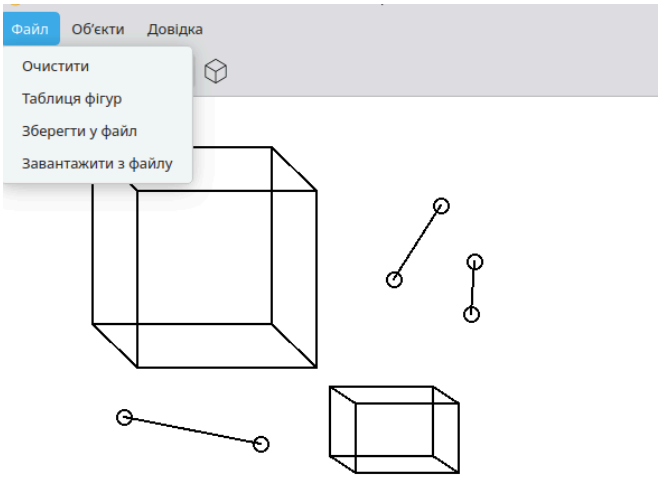
    QString getInfo() const override;
    void createFromInfo(const QString& info) override;
    QString getCoordinates() const override;
    QString getShapeName() const override { return "Куб"; }
};
```

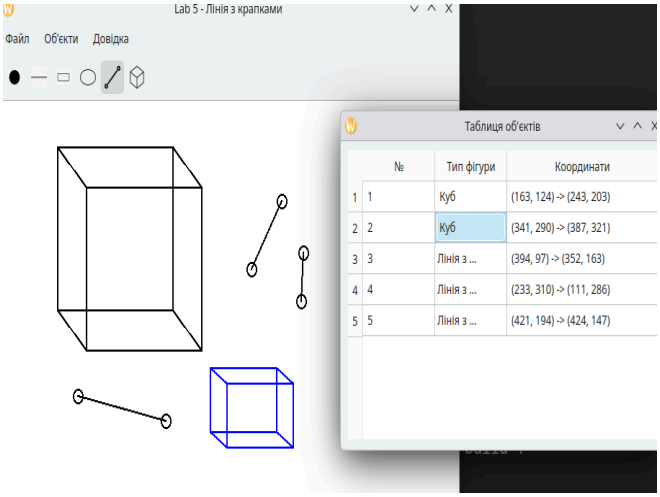
#endif // SHAPE_H

Скріншоти

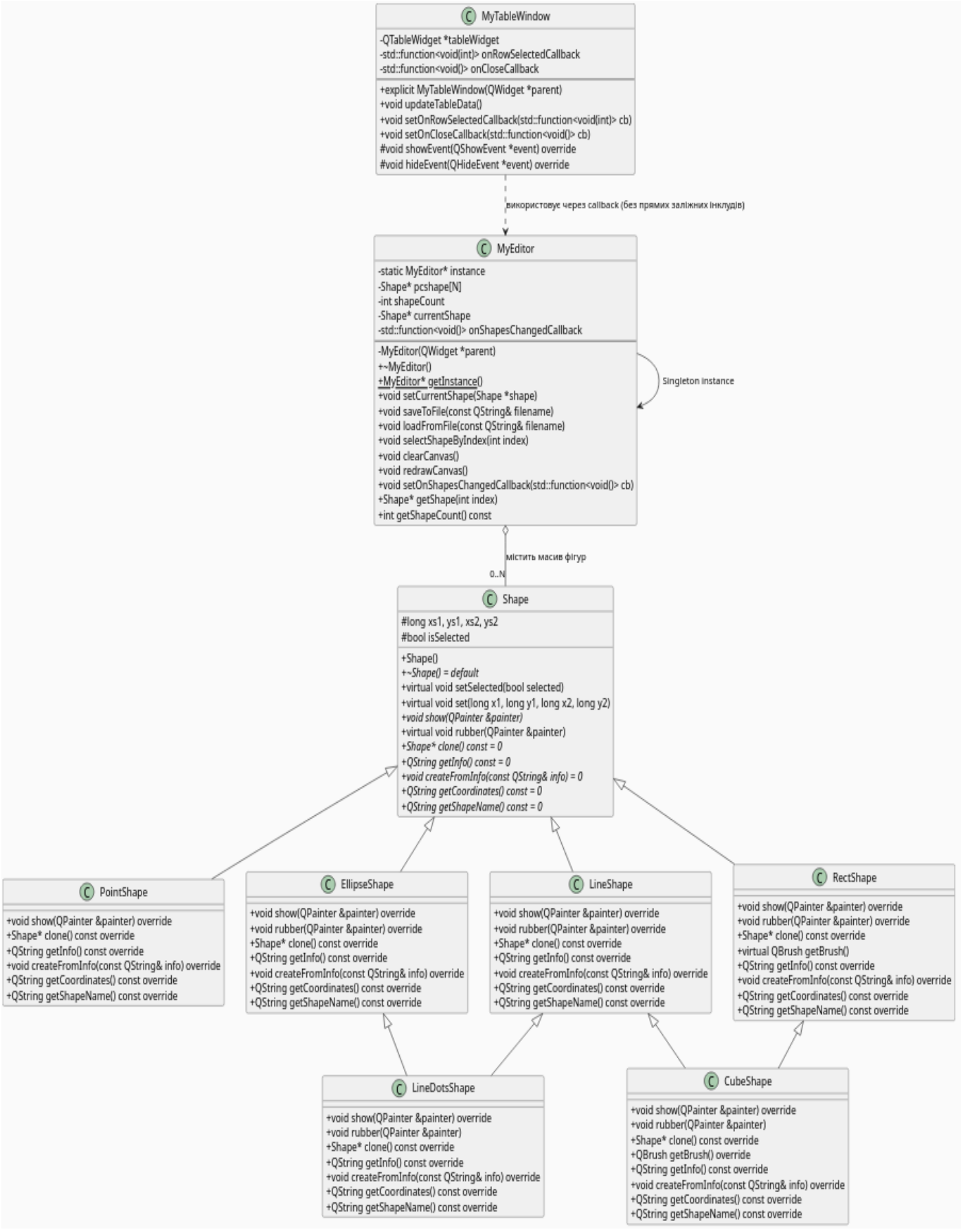








Діаграма залежностей класів



Висновки:

У ході виконання лабораторної роботи 5 успішно реалізовано немодальне вікно таблиці та поліморфне збереження й завантаження фігур за допомогою патернів Singleton і Callback. Завдяки віртуальним методам та динамічному зв'язуванню досягнуто повної синхронізації стану між графічним редактором та таблицею в реальному часі. Застосування архітектурних розв'язок дозволило уникнути зайвих модульних залежностей і забезпечити стабільну роботу програми без помилок пам'яті.